

• Section-A ($5 \times 2 = 10$)

1. Define the terms in-degree and out-degree in the context of directed graph. How are these measures used in practical applications.

Ans In a directed graph:

- In-degree of a node is the number of edges directed towards it.
- Out-degree of a node is the number of edges directed away from it.

These measures help analyze network flow, influence and connectivity in applications like social networks (e.g. follower counts), web pages ranking and traffic routing.

2. If a graph has 10 vertices and a total of 15 edges, what is the minimum no. of edges that must be removed to form a spanning tree.

Ans For a graph with n vertices, a spanning tree contains exactly $n-1$ edges.

So here 10 vertices

$$\text{edges} = 10 - 1 = 9 \text{ edges}$$

minimum no. of edges that must be removed

$$\Rightarrow 15 - 9 = 6$$

3. Given a text string of length 100 characters and a pattern of length 5 characters, how many character comparisons would be needed in the worst case for the brute force string matching algo to find all occurrences of the pattern in the text.

Ans (length of the text (n)) = 100

length of the pattern (m) = 5

The algo performs $(n - m + 1) \times m$ comparisons

$$\Rightarrow (n - m + 1) \times m$$

$$\Rightarrow (100 - 5 + 1) \times 5$$

$$\Rightarrow 96 \times 5$$

$$= 480$$

4. What is the hash value of the pattern 'ABCD' using a simple hash function, where 'A' represents 1, 'B' represents 2, 'C' represents 3, and 'D' represents 4.

Ans Hash value = $1+2+3+4$
 $= 10$

5. What is the primary objective of the KMP algorithm and how does it achieve this?

Ans KMP algorithm is a string searching algorithm which is used to find a pattern within large text efficiently. KMP uses the structure of the pattern to avoid redundant comparisons. It preprocesses the pattern string and creates an array called the Longest Prefix Suffix (LPS) array which indicates how much pattern can be returned after a mismatch.

• Section - B ($9 \times 7 = 21$)

6. Describe the concept of an "adjacency list", in the context of graph representation. Explain how an adjacency list is used to represent a graph and discuss the advantages and disadvantages of using adjacency lists compared to other graph representation methods.

Ans An adjacency list is a data structure used to represent a graph. It consists of an array of lists where each list stores the vertices adjacent to a particular vertex.

• Advantages:-

- i) Efficient for sparse graph.
- ii) Dynamically add/remove vertices and edges.
- iii) Natural for many graph algorithms.

• Disadvantages:-

- i) Less efficient for dense graph.
- ii) Slower to check edge existence.

Scenario: Social Networks

In a social network application, users (represented as vertices) are connected to a friendships (edges).

The graph is typically sparse because not all users are friends with each other.

7. Given a text string "ABCD CBA ABDCBA" and a pattern "AB", find and list all occurrences of the patterns using the naive algo. Provide the position of the occurrence within the text.

Ans

A B C D C B A A B D C B A A

Index: 0 1 2 3 4 5 6 7 8 9 10 11 12 13

At index: 0 $\rightarrow$ Match "AB"

At index: 1, 2, 3, 4, 5, 6 $\rightarrow$ does not match.

At index: 7 $\rightarrow$ matches "AB"

At index: 8, 9, 10, 11, 12, 13 $\rightarrow$ does not match

The pattern 'AB' occurs at index: 0, 7

8. Explain the concept of the Rabin-Karp algo. and its primary use. Given a text "ABCD ABCD ABCD" and a pattern "ABC", calculate the hash value of the pattern and determine the position of pattern occurrence using the Rabin-Karp algo.

Ans Rabin-Karp Algorithm:- It uses a rolling hash function to efficiently find pattern occurrence. It calculates the hash value of the pattern and each window in the text. If hash value matches then a potential match is found and verified. It is efficient for multiple pattern occurrences.

Text: ABCD ABCD ABCD

Pattern: ABC

$$\text{hash}(\text{ABC}) = (1 \times 10^2 + 2 \times 10^1 + 3 \times 10^0) \% 13$$

$$= 6$$

hash value of first window 'ABC' = 6 (matched)
pattern found at position 0.

To calculate hash value of next window 'BCD'

$$\text{new-hash} = [(old\ hash - first\ char \times base^{(m-1)}) \times base + last\ char] \% 13$$

$$\text{new-hash} = [(6 - 1 \times 10^2) \times 10 + 4] \% 13 = 8 \text{ (not matched)}$$

Final Occurance

position: 0

position: 4

position: 8

• Section - C (3x11=33)

(9)(a) Explain the key principles of the finite automata string matching algo including its main components and discuss the strength and limitations of finite automata for string matching compared to other algo.

Ans The finite automata string matching algorithm is a deterministic approach to finding patterns within a text string. It constructs a finite automata, a mathematical model of a machine that can be in one of a finite number of states:

key principles:-

- i) state transition diagram
- ii) start state
- iii) Accepting state
- (iv) Transition function

• Strengths:-

- Efficient Matching
- Deterministic: No backtracking needed

Limitations :-

- Preprocessing overhead
- High space complexity
- Less flexible.

(b) Using the provided finite automaton, find all occurrences of the pattern "ABAB" in the text "ABABABABABABABABAB" and show the state transitions. Describe how the finite automaton processes the text to detect the pattern, including state transitions and the final state it reaches.

Ans Text: ABABABABABABABABAB
Pattern: ABAB

Finite automaton: $A \rightarrow B \rightarrow A \rightarrow B$ (Accept)

State transition:-

- Initial state:- The automaton starts in the initial state (A)
- First 'A' in the text: the automaton transition to state B.
- Second 'B' in the text: The automaton transition to state A.
- Third 'A' in the text: The automaton transition to state B.
- Fourth 'B' in the text: The automaton reaches the accepting state (B) indicating a match of the pattern 'ABAB'.

Pattern found at index 0

Pattern found at index 2

Pattern found at index 4

Pattern found at index 6

Pattern found at index 8

(10) (a) Explain the key principle of Knuth-Morris-Pratt (KMP) string matching algorithm. Include its preprocessing phase and pattern matching phase. Provide a step-by-step explanation of how the algorithm works and its time complexity.

the
AAB

Key steps:

- i) Preprocessing: Calculate the Prefix function for the pattern
- ii) Pattern Matching:
 - Compare characters
 - If mismatch, shift the Pattern using the Prefix function
 - Continue until a match or end of text.

Time Complexity : $O(n+m)$ (Linear time)

- (b) Perform the preprocessing steps for the pattern "ABAB" and calculate the partial match table. Then use the calculated table to apply the KMP algo and find all occurrences of the pattern "ABAB" in the text "ABABABABBBBABABABA." Show the position and state transitions for each occurrence.

Ans Preprocessing Phase :
pattern : ABAB

Index	Character	Longest Proper Prefix Suffix
0	A	0
1	B	0
2	A	1
3	B	2

match found at index: 0, 2, 4, 9, 11

(11) (a) Explain the key principles of Breadth-First Search (BFS) and Depth-First Search (DFS) in graph traversal. Discuss the fundamental differences between these two algo. and provide a real-world scenario when choosing one algorithm over the other is more advantageous.

Ans

Breadth-First-Search (BFS):

- Explores graph Level by Level
- Uses a queue
- Find shortest path in unweighted graphs.

Depth-First-Search (DFS):

- Explores graph depth first
- Uses a stack
- Used for topological sorting, cycle detection.

Differences:

	BFS	DFS
i) Data Structure	Queue	Stack
ii) Search order	Breadth First	Depth First
iii) Time complexity	$O(V+E)$	$O(V+E)$
iv) space complexity	$O(V)$	$O(V)$ in worst case

Scenario: -

- BFS: Ideal for finding shortest path in unweighted graph.
- DFS: Useful for topological sorting and cycle detection.

(11) (B) Explain the key principles of the Boyer-Moore string Matching algorithm, including its components and advantages compared to naive S.M. algo. Given a text "ABCDABCDABCD" and a pattern "ABC". Calculate the number of character comparisons made when finding the pattern.

Ans The Boyer-Moore algorithm is a highly efficient string searching algo that leverage two key ~~principles~~ techniques to minimize the no. of character comparisons.

ii) Bad character Heuristic: Examines the mismatched character in the pattern and the corresponding character in the text. Shifts the pattern to the right, aligning the mismatched character with the next occurrence of that character in the pattern.

iii) Good suffix Heuristic: Examines the portions of the pattern that has matched so far and shifts the pattern to the right, aligning the longest suffix of the matched portion within the corresponding suffix in the text.

• Advantages over Naive String Matching :

- ↳ Efficiency
- ↳ Reduced characters comparisons.

text : ABCD ABCD ABCD

Pattern : ABC

Pattern occurs at shift 0, 4, 8.

